



資料結構

Data Structure

Lab 11

姓名: 盧敬恩
學號: 114310123

Lab_11-Ex1

完成BFS/LEVEL SUM

Code

```

#include <iostream>
#include <vector>
#include <queue>

using namespace std;

// 定義二元樹的節點結構
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {} //這裡和之前所有的TREE都一樣
};

class BinaryTree {
public:
    TreeNode* root;

    BinaryTree() : root(nullptr) {}

    // 講義的NULL 改成0
    void buildTree(const vector<int>& arr) {
        if (arr.empty() || arr[0] == 0) { //這裡對應NULL 改成0 當NODE = 0不會BUILD
            root = nullptr;
            return;
        }

        root = new TreeNode(arr[0]);
        queue<TreeNode*> q;
        q.push(root);

        int i = 1;
        while (!q.empty() && i < arr.size()) {
            TreeNode* current = q.front();
            q.pop();

            if (i < arr.size() && arr[i] != 0) {
                current->left = new TreeNode(arr[i]);
                q.push(current->left);
            }
            i++;

            if (i < arr.size() && arr[i] != 0) {
                current->right = new TreeNode(arr[i]);
                q.push(current->right);
            }
            i++;
        }
    }

    // 1.BFS: 輸出整棵樹的走訪結果 把該層走完再下一層
    void Breadth_first_search(TreeNode* node) {
        if (node == nullptr) return;
        queue<TreeNode*> q; //存放節點的佇列
        q.push(node);

        while (!q.empty()) { //當QUENE還有東西 = 有節點
            TreeNode* current = q.front(); //當前節點為Q FRONT
            q.pop(); //把當前節點踢掉
            cout << current->val << " ";
            if (current->left != nullptr) q.push(current->left); //先把他的左子TREE加入QUENE
            if (current->right != nullptr) q.push(current->right); //再把他的右子TREE加入QUENE
        }
    }
};

```

```

}

void Breadth_first_search_LevelSum(TreeNode* root, int layer) { //當查找<0或是父TREE 指向NULL表示不存在
    if (root == nullptr || layer < 0) {
        cout << "The layer exceeds the tree height." << endl; //輸出不存在
        return;
    }

    queue<TreeNode*> q;
    q.push(root);

    int level = 0;
    int levelsum = 0;
    bool isLayerFound = false;

    while (!q.empty()) {
        int levelsize = q.size();
        //這部分是分層遍歷 迴圈執行次數=節點個數
        for (int i = 0; i < levelsize; ++i) {
            TreeNode* curr = q.front();
            q.pop();

            if (level == layer) { //當當前LEVEL = 我要找的
                levelsum += curr->val;
                isLayerFound = true; //代表找到了我要的SUN LEVEL
            }
            else {
                if (curr->left != nullptr) q.push(curr->left);
                if (curr->right != nullptr) q.push(curr->right);
            }
        }

        if (level == layer) { //這裡要等到FOR LOOP結束 不然QUEUE中的東西還沒加總
            cout << "The sum of level " << layer << " is: " << levelsum << endl;
            return;
        }

        level++;
    }

    if (!isLayerFound) { //如果跑了上面的LOOP都沒找到代表超出範圍
        cout << "The layer exceeds the tree height." << endl;
    }
}

};

int main() {
    BinaryTree tree;
    // 使用 0 代表 NULL
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 0, 10, 11, 0, 0 };
    tree.buildTree(arr);

    // 先輸出完整的 BFS 結果
    cout << "BFS Result: ";
    tree.Breadth_first_search(tree.root);
    cout << endl;

    // 提示使用者輸入層數
    int targetLayer;
    cout << "Please enter the layer to query, starting from 0: ";
    cin >> targetLayer;

    // 執行 BFS 計算層數總和與判斷超過邏輯
    tree.Breadth_first_search_LevelSum(tree.root, targetLayer);

    system("pause");
    return 0;
}

```

Result

C:\Users\user\Desktop\新增頁 ×

+

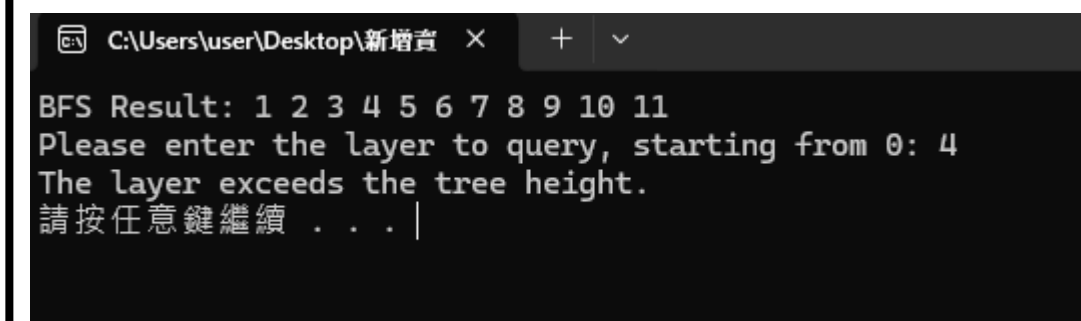
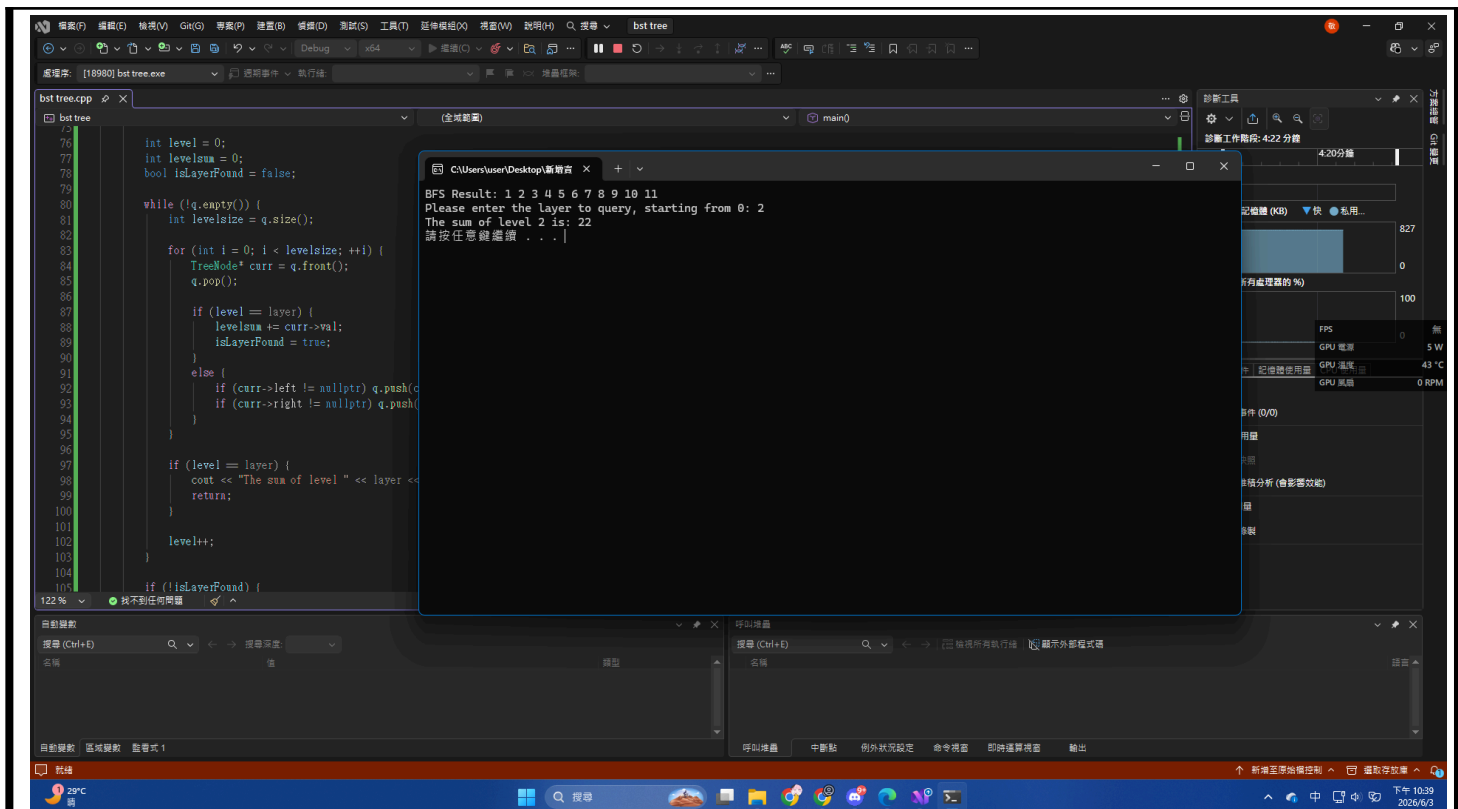
▼

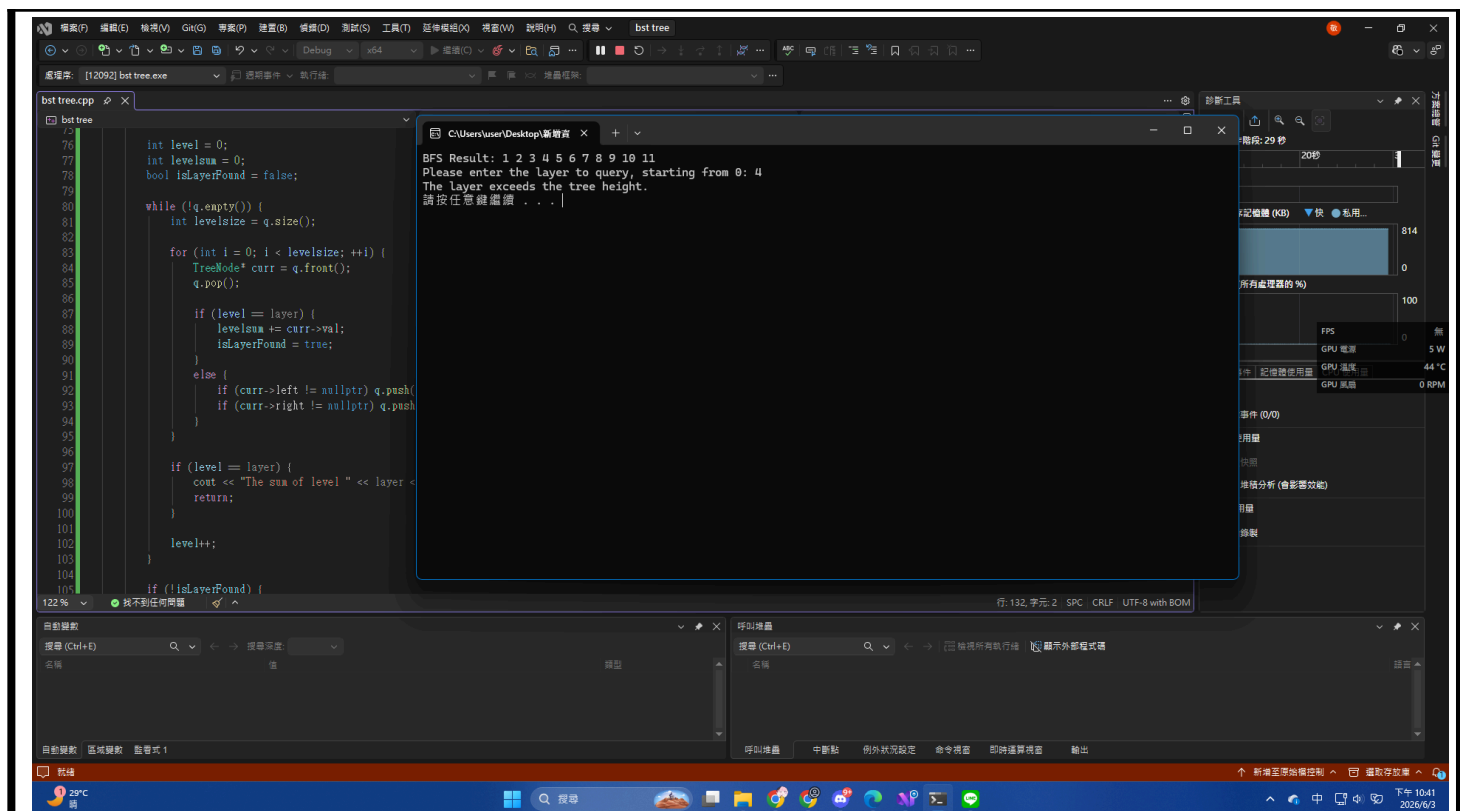
BFS Result: 1 2 3 4 5 6 7 8 9 10 11

Please enter the layer to query, starting from 0: 2

The sum of level 2 is: 22

請按任意鍵繼續 . . . |





Discussion

這次的實驗更像是之前BINARY TREE增加一種LEVEL 尋訪方式